PHYSICAL AI AND SCIML IN THREE GOES · HANDBOOK

Go 2: Build and Implement

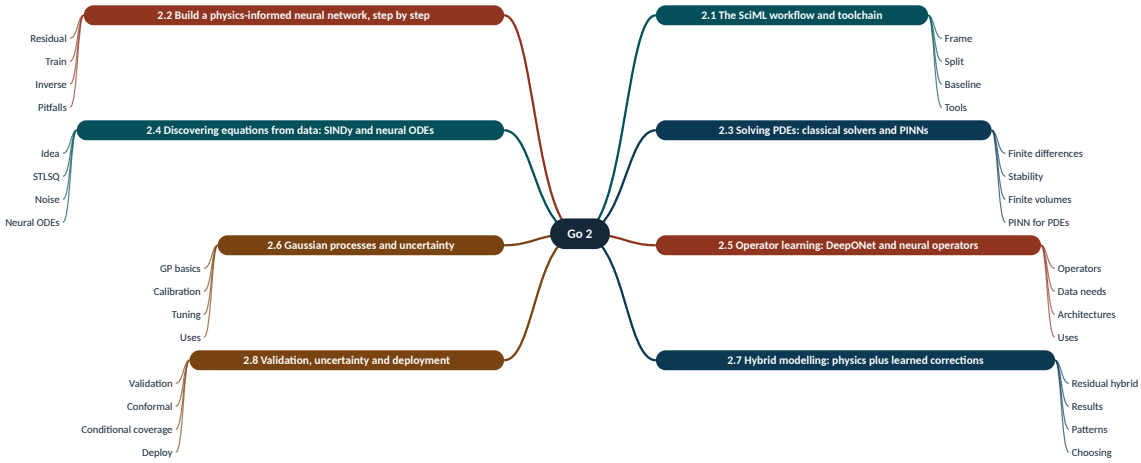
8 modules · about 6.4 hours of study

Companion to the website. Explorers, simulations, challenges and runnable code are online; this handbook keeps the text, worked numbers, mind maps and questions for reading offline.

Contents

1. **2.1 The SciML workflow and toolchain** · 45 min
   A reliable SciML project follows a fixed workflow: frame the question, gather data and physics, split data honestly, build a baseline, add physics, validate and document. Tools: Python, NumPy, PyTorch or JAX.
2. **2.2 Build a physics-informed neural network, step by step** · 55 min
   Build a PINN for an ODE from scratch: define the network, compute the residual by automatic differentiation, assemble the loss, train with Adam, watch both loss terms, and solve an inverse problem.
3. **2.3 Solving PDEs: classical solvers and PINNs** · 50 min
   Partial differential equations are solved on grids by finite differences or volumes, with stability limits on the time step. PINNs solve them without grids. Learn both, and when each wins.
4. **2.4 Discovering equations from data: SINDy and neural ODEs** · 50 min
   Instead of assuming the law, find it. SINDy fits a sparse combination of candidate terms; neural ODEs learn the right-hand side of a differential equation with a network.
5. **2.5 Operator learning: DeepONet and neural operators** · 50 min
   An operator maps a whole input function to a whole output function. Once trained on example pairs, a neural operator solves new cases of a PDE in milliseconds, across many inputs at once.
6. **2.6 Gaussian processes and uncertainty** · 45 min
   A Gaussian process predicts a value and its uncertainty, widening its error bars far from data. Kernels encode smoothness; marginal likelihood tunes them; calibrated uncertainty is the goal.
7. **2.7 Hybrid modelling: physics plus learned corrections** · 45 min
   A hybrid model keeps a physics model and learns only what it gets wrong. It needs fewer data than pure machine learning and extrapolates far more safely, as long as the physics part is sound.
8. **2.8 Validation, uncertainty and deployment** · 45 min
   Before a SciML model is used, test it on unseen conditions, check physical consistency, attach calibrated intervals such as conformal predictions, and plan for monitoring, drift and fallbacks in deployment.

Mind map of the go



2.1 The SciML workflow and toolchain

45 minutes · Workflow

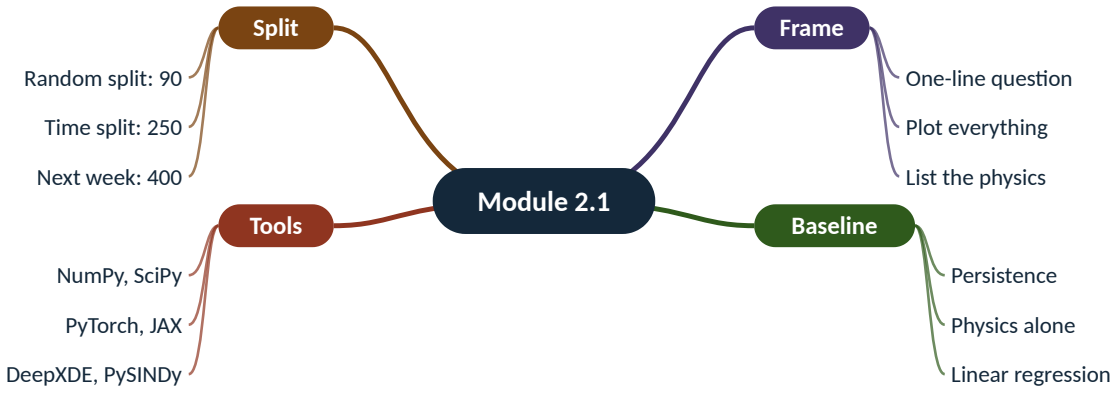
A reliable SciML project follows a fixed workflow: frame the question, gather data and physics, split data honestly, build a baseline, add physics, validate and document. Tools: Python, NumPy, PyTorch or JAX.

By the end you can

- lay out the steps of a SciML project from question to deployment
- set up a Python environment with NumPy, PyTorch or JAX
- split time-series data without leakage
- build a simple baseline before any neural network

Overview

Go 2 is about building. Before writing a single network, set up a workflow that protects you from the most common mistakes. Step one: frame the question precisely, including what you will predict, from what inputs, and how you will judge success. Step two: gather the physics you trust and the data you have, and look at the data. Step three: split the data honestly. For time series, such as traffic counts, split by time: train on the past, validate on later data. A random split lets the model peek at hours next to the ones it is tested on, and reports an error far better than you will get in practice. Step four: build a simple baseline, such as a physics model alone or a linear regression, so you know what a fancy model must beat. Step five: add learning and physics, step by step. Step six: validate, quantify uncertainty and document everything so others can reproduce it. The toolchain is Python with NumPy, plus PyTorch or JAX for automatic differentiation. In this module you will see how a random split fools you.



Step 1–2: question, data and physics

Write a one-line question, for example: *predict hourly traffic flow at detector D for the next 24 hours from the past four weeks, scored by mean absolute error*. Then plot the data before modelling: daily and weekly patterns, missing hours, sensor failures. List the physics you trust (conservation of vehicles, capacity limits) and the parameters you do not know.

Step 3: split without leakage

Leakage is information from the test set sneaking into training. In the explorer, a forecaster that averages the three nearest known hours reports an error of about 90 veh/h with a random split but about 250 veh/h with a time split; its real error next week is about 400 veh/h. The time split is honest; the random split is not. Rules: split time series by time, spatial data by site, and fit any normalisation on training data only.

TRY Try it

Switch the explorer between random and time splits. Which validation error is closer to the real error next week?

Step 4: baselines

Always build the simplest reasonable model first: yesterday's value at the same hour, a physics model alone, or a linear regression. Report your SciML model's improvement over this baseline. Many published deep-learning results shrink or vanish against a good baseline.

The toolchain

Python with **NumPy** and **SciPy** for arrays and classical solvers; **PyTorch** or **JAX** for neural networks and automatic differentiation; **DeepXDE**, **NVIDIA PhysicsNeMo** (formerly Modulus) or **NeuroMANCER** for physics-informed models; **PySINDy** for equation discovery; **scikit-learn** or **GPyTorch** for Gaussian processes. Use a virtual environment, pin versions, and fix random seeds. Every Go 2 lab has a NumPy version that runs in your browser and, where useful, a PyTorch version to download.

MODEL Setting up

A minimal environment: `python -m venv sciml`, then `pip install numpy scipy matplotlib torch`. Save it with `pip freeze > requirements.txt`.

LAB Random versus time-based splits
`labs/go2/m21_workflow.py`

The same forecaster, two validation schemes, and the error it actually achieves next week.

```
random split: validation error   106 veh/h, real error next week   424 veh/h
time   split: validation error   272 veh/h, real error next week   424 veh/h
```

Takeaways

- Write the question and the success metric before modelling.
- Split time series by time and spatial data by location to avoid leakage.
- Always compare against a simple baseline.
- Use Python with NumPy plus PyTorch or JAX; pin versions and seeds.

Check yourself

1. For traffic time series, the honest split is…
   (a) Random hours · (b) By time: train on the past, validate on later data · (c) By alphabetical detector name · (d)

No split

2. Leakage means…
   (a) Data lost from a disk · (b) Test information sneaking into training · (c) Too few parameters · (d) A slow model
3. Why build a baseline first?
   (a) To save time later · (b) To know what a complex model must beat · (c) Because networks need it · (d) It is required by PyTorch
4. Which library provides automatic differentiation?
   (a) Matplotlib · (b) PyTorch · (c) Pandas · (d) Requests

Answers: 1 (b) Mimics real forecasting.; 2 (b) The model peeks at the answer.; 3 (b) Sets the bar.; 4 (b) PyTorch and JAX.

Words

- **Data leakage**: Information from validation or test data influencing training, giving over-optimistic results.
- **Baseline**: A simple reference model that a new method must outperform.
- **Train/validation/test split**: Dividing data for fitting, tuning and final evaluation.
- **Reproducibility**: The ability of others to rerun an analysis and get the same results.
- **Virtual environment**: An isolated Python installation with pinned package versions.

Reading

- Kapoor and Narayanan (2023): Leakage and the reproducibility crisis in ML-based science: Leakage across 17 fields. https://doi.org/10.1016/j.patter.2023.100804
- DeepXDE documentation: A library for physics-informed learning with many examples. https://deepxde.readthedocs.io/
- PyTorch tutorials: Official beginner tutorials. https://pytorch.org/tutorials/

2.2 Build a physics-informed neural network, step by step

55 minutes · Build

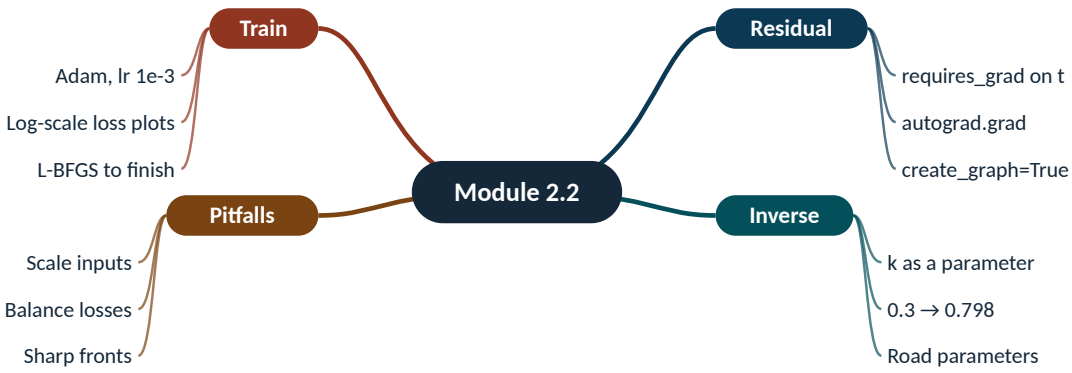
Build a PINN for an ODE from scratch: define the network, compute the residual by automatic differentiation, assemble the loss, train with Adam, watch both loss terms, and solve an inverse problem.

By the end you can

- implement a PINN for a first-order ODE in NumPy and in PyTorch
- compute the physics residual with automatic differentiation
- monitor data and physics losses to diagnose training
- estimate an unknown parameter (inverse problem) with a PINN

Overview

Now we build a physics-informed neural network ourselves, step by step. Our problem: a quantity decays as dy over dt equals minus k y, we have six noisy measurements in the first second, and we want the whole curve to five seconds. Step one, define the network: one or two hidden layers of tanh neurons mapping time to y. Step two, choose collocation points, say forty times spread over zero to five seconds. Step three, compute the residual: the network's derivative with respect to time plus k times its output. In PyTorch, one call to autograd gives the derivative. Step four, build the loss: data error, plus the physics residual, plus the initial condition. Step five, train with the Adam optimiser and plot both loss terms; if one stays high, rebalance. Step six, the payoff: make k itself a trainable parameter, and the PINN discovers the decay rate from the data. Our NumPy version, with hand-derived gradients, recovers k as 0.798 when the true value is 0.8. In the explorer you will watch the two losses fall during training.



Steps 1–3: network, points, residual

In PyTorch:

```
net = nn.Sequential(nn.Linear(1,32), nn.Tanh(), nn.Linear(32,32), nn.Tanh(),
nn.Linear(32,1))
tc = torch.linspace(0,5,40).reshape(-1,1).requires_grad_(True)
```

```
y = net(tc); dy = torch.autograd.grad(y, tc, torch.ones_like(y),
create_graph=True)[0]
residual = dy + k*y
```

`create_graph=True` lets the optimiser differentiate through the derivative. Use smooth activations (tanh, not ReLU), because ReLU's second derivative is zero.

Step 4–5: loss and training

`loss = mse(net(td), yd) + lam*mean(residual**2) + mse(net(0), 1)`, then `loss.backward(); opt.step()` with `torch.optim.Adam(..., lr=1e-3)`. Plot the data and physics losses separately on a log scale. Healthy training: both fall. Data loss stuck high: physics weight too big or network too small. Physics loss stuck high: weight too small or too few collocation points. Many practitioners finish with the L-BFGS optimiser for extra accuracy.

TRY Try it

Set $\lambda = 10^{-3}$ and train. Which loss stays high? Now try $\lambda = 10^{2}$. What happens to the data loss?

Step 6: the inverse problem

Make k trainable: `k = nn.Parameter(torch.tensor(0.3))` and add it to the optimiser. Now the PINN looks for a network *and* a k such that the data fit and the law holds. In the NumPy lab (hand-derived gradients, 16 neurons, 8,000 Adam steps) k moves from 0.3 to 0.798, and the error beyond the data is under 0.001. This is how PINNs estimate unknown physical parameters, such as a road's jam density, from sparse measurements.

RESEARCH Research angle

Inverse problems are where PINNs beat classical methods most clearly: they fuse sparse data and physics to estimate parameters that cannot be measured directly.

Common failure modes

Unscaled inputs: scale t and y to about −1…1. **Imbalanced losses**: rescale or use adaptive weights. **Long time ranges or sharp fronts**: PINNs struggle (spectral bias, causality violations); split the domain or train sequentially in time. **Too few collocation points**: the law is not enforced between them. Always compare with a classical solver when one exists.

LAB A PINN from scratch in NumPy (with the inverse problem)
`labs/go2/m22_pinn.py`

A one-hidden-layer tanh network; the gradients of the data and physics losses are derived by hand, and k is learned.

```
epoch     1: data loss 1.22e+00, physics loss 5.74e-02, k = 0.290
epoch  2000: data loss 1.73e-05, physics loss 1.41e-05, k = 0.797
epoch  8000: data loss 1.30e-05, physics loss 4.56e-07, k = 0.798
error beyond the data: 0.0007
```

LAB The same PINN in PyTorch

`labs/go2/m22_pinn_torch.py`

Autodiff replaces the hand-derived gradients. Requires pip install torch; about 10 seconds on a laptop CPU.

```
epoch 2000: data 1.05e-04, physics 9.91e-06, k = 0.827
epoch 4000: data 1.05e-04, physics 1.11e-06, k = 0.828
epoch 6000: data 1.04e-04, physics 1.33e-06, k = 0.828
error beyond the data: 0.0081
```

Takeaways

- A PINN needs a smooth network, collocation points, an autodiff residual and a composite loss.
- Train with Adam and monitor data and physics losses separately.
- Making a physical parameter trainable turns a PINN into an inverse-problem solver.
- Scale inputs, balance losses and compare with classical solvers.

Check yourself

1. Why use tanh rather than ReLU in a PINN?
   (a) It is faster · (b) Its derivatives are smooth and nonzero, which the residual needs · (c) ReLU is not allowed in PyTorch · (d) No reason
2. What does create_graph=True do?
   (a) Draws a plot · (b) Lets the optimiser differentiate through the computed derivative · (c) Creates a new network · (d) Saves the model
3. The physics loss stays high while data loss is tiny. Try…
   (a) Lowering λ · (b) Raising λ or adding collocation points · (c) Removing the data · (d) Using ReLU
4. To solve an inverse problem with a PINN you…
   (a) Remove the physics term · (b) Make the unknown parameter trainable · (c) Reverse time · (d) Use more layers only

Answers: 1 (b) Residuals need derivatives.; 2 (b) Higher-order autodiff.; 3 (b) Physics is losing.; 4 (b) Learn k with the network.

Words

- **Residual**: The amount by which the network breaks the governing equation at a point.
- **Adam**: An adaptive gradient-based optimiser widely used for training PINNs.
- **L-BFGS**: A quasi-Newton optimiser often used to refine PINN training.
- **Spectral bias**: The tendency of networks to learn smooth, low-frequency features first.
- **Inverse problem**: Estimating unknown parameters of a model from data.

Reading

- Raissi, Perdikaris and Karniadakis (2019): Original PINN paper with inverse examples. https://doi.org/10.1016/j.jcp.2018.10.045
- Lu et al. (2021): DeepXDE: A library and a tutorial paper for PINNs. https://doi.org/10.1137/19M1274067
- Wang, Sankaran and Perdikaris (2024): Respecting causality for training PINNs: Fixing a common failure mode. https://doi.org/10.1016/j.cma.2024.116813

2.3 Solving PDEs: classical solvers and PINNs

50 minutes · Build

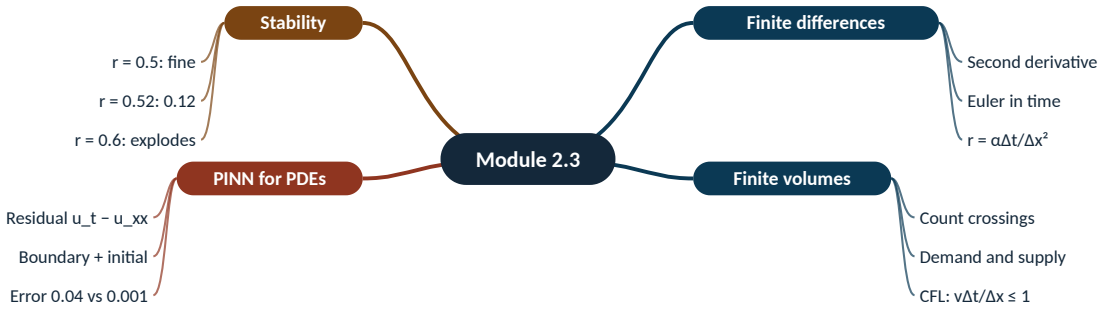
Partial differential equations are solved on grids by finite differences or volumes, with stability limits on the time step. PINNs solve them without grids. Learn both, and when each wins.

By the end you can

- discretise the heat equation with finite differences
- explain and check the stability condition r ≤ 0.5
- set up a PINN for a PDE with boundary and initial conditions
- compare classical solvers and PINNs fairly

Overview

Many systems vary in space as well as time: temperature in a pavement, pollution along a street, traffic density along a road. They obey partial differential equations. The classical way to solve them is to cut space and time into a grid. For the heat equation, the rate of change at a point is proportional to how much its neighbours differ from it, so each step updates every grid point from its neighbours. This explicit finite-difference scheme is simple, but it has a stability limit: the number r, the diffusivity times the time step divided by the square of the grid spacing, must stay at or below one half. Cross it and small wiggles grow until the solution explodes. For traffic, conservation laws are solved with finite-volume schemes such as Godunov's. A PINN takes a different route: it needs no grid, only collocation points in space and time, plus boundary and initial conditions in the loss. For simple forward problems a classical solver is usually faster and more accurate, but PINNs handle odd geometries, missing information and inverse problems. In this module you will push a solver past its stability limit.



Finite differences for heat

Approximate the second derivative by neighbours: $\partial^2 u/\partial x^2 \approx (u_{i+1} - 2u_i + u_{i-1})/\Delta x^2$. With an Euler step in time:

$$u_i^{n+1} = u_i^n + r\,(u_{i+1}^n - 2u_i^n + u_{i-1}^n),\ r = \alpha\Delta t/\Delta x^2$$

A hot point loses heat to cooler neighbours; a cold point gains. Ends are held at zero (boundary conditions).

Stability: the r ≤ 0.5 rule

The explicit scheme is stable only for $r \le 0.5$ (von Neumann analysis). In the lab with 41 points: $r = 0.25$ and $r = 0.5$ give errors below 0.0004, $r = 0.52$ gives 0.12, and $r = 0.6$ explodes to 10^7. Halving Δx forces a quarter of the time step, which is why implicit schemes (Crank–Nicolson) are used for fine grids.

TRY Try it

Set $r = 0.49$, then 0.51. Look closely at the profile: where do the first wiggles appear?

Traffic: finite volumes

Conservation laws such as the LWR traffic model have shock waves (sudden jams), where finite differences oscillate. **Finite-volume** schemes track how many vehicles cross each cell boundary; **Godunov's scheme** (in traffic, the cell transmission model of Daganzo 1994) takes the smaller of upstream demand and downstream supply. Module 3.1 uses it as the physics inside a traffic state estimator. Its time step obeys the CFL condition: $v_f \Delta t / \Delta x \le 1$.

A PINN for the heat equation

Network $u(x, t)$; loss = mean $(u_t - u_{xx})^2$ at 1,000 random points, plus boundary terms $u(0,t)^2 + u(1,t)^2$, plus initial term $(u(x,0) - \sin \pi x)^2$. The PyTorch lab trains in about 20 seconds and reaches a maximum error of about 0.04 at $t = 0.1$, while the finite-difference solver gets below 0.001 in a fraction of a second. For this forward problem the classical solver wins; PINNs earn their place when data must be fused or parameters inferred.

TRAP Fair comparison

Papers sometimes compare a carefully tuned PINN with a crude classical solver. Compare at equal accuracy or equal computing time.

LAB Heat equation solver and stability check

`labs/go2/m23_heat.py`

Explicit finite differences for four values of r.

```
r = 0.25: 192 steps, max error 7.10e-05
r = 0.50:  96 steps, max error 3.68e-04
r = 0.52:  93 steps, max error 1.20e-01  <- unstable
r = 0.60:  80 steps, max error 4.75e+07  <- unstable
```

LAB A PINN for the heat equation (PyTorch)

`labs/go2/m23_pinn_heat_torch.py`

Physics, boundary and initial losses only; no data inside the domain. Requires pip install torch; about 20 seconds.

```
iteration 4000: loss 2.51e-03
max error at t = 0.1: 0.0402
```

Takeaways

- Grid solvers update each point from its neighbours.
- Explicit finite differences for heat need $r = \alpha\Delta t/\Delta x^2 \le 0.5$.
- Finite-volume (Godunov) schemes handle traffic shock waves under a CFL limit.
- PINNs need no grid; classical solvers usually win simple forward problems.

Check yourself

1. The stability number for the explicit heat scheme is…
   (a) $\alpha\Delta x/\Delta t$ · (b) $\alpha\Delta t/\Delta x^2$ · (c) $\Delta t/\alpha$ · (d) Δx^2
2. The explicit scheme is stable if…
   (a) $r \le 0.5$ · (b) $r \ge 1$ · (c) $r = 2$ · (d) Always
3. Which scheme suits traffic shock waves?
   (a) Central finite differences · (b) Godunov finite-volume (cell transmission model) · (c) Polynomial fitting · (d) None
4. For a simple forward heat problem, a PINN is usually…
   (a) Faster and more accurate than finite differences · (b) Slower and less accurate than a good classical solver · (c) Identical · (d) Impossible

Answers: 1 (b) $r = \alpha\Delta t/\Delta x^2$.; 2 (a) Von Neumann analysis.; 3 (b) Demand and supply.; 4 (b) Classical solvers win there.

Words

- **Finite differences**: Approximating derivatives by differences between neighbouring grid values.
- **Finite volumes**: Tracking conserved quantities by fluxes across cell boundaries.
- **Stability condition**: A limit on step sizes beyond which numerical errors grow without bound.
- **CFL condition**: Courant–Friedrichs–Lewy limit: information must not travel more than one cell per step.
- **Boundary condition**: A constraint on the solution at the edges of the domain.

Reading

- LeVeque: Finite Volume Methods for Hyperbolic Problems: The standard text on shocks and Godunov schemes. https://doi.org/10.1017/CBO9780511791253
- Daganzo (1994): The cell transmission model: Godunov's scheme for traffic. https://doi.org/10.1016/0191-2615(94)90002-7
- Grossmann et al. (2024): Can PINNs beat the finite element method?: A careful, fair comparison. https://doi.org/10.1093/imamat/hxae011

2.4 Discovering equations from data: SINDy and neural ODEs

50 minutes · Build

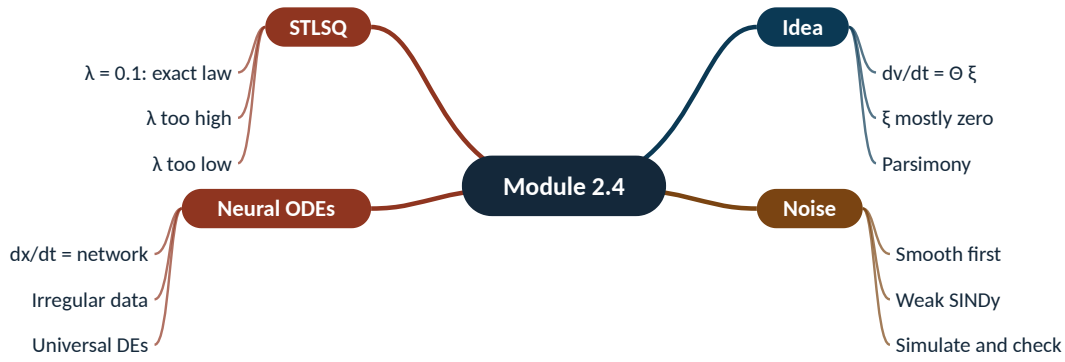
Instead of assuming the law, find it. SINDy fits a sparse combination of candidate terms; neural ODEs learn the right-hand side of a differential equation with a network.

By the end you can

- explain sparse regression for equation discovery (SINDy)
- build a candidate library and apply sequential thresholding
- describe how noise and the threshold affect what is found
- explain the idea of a neural ODE and when to prefer it

Overview

So far we assumed we knew the law. Sometimes we do not. Equation discovery tries to find it from data. The SINDy method, short for sparse identification of nonlinear dynamics, by Brunton, Proctor and Kutz, rests on a simple observation: most physical laws contain only a few terms. So we measure the state, estimate its time derivatives, and build a library of candidate terms, such as a constant, x, v, x squared, x times v and x cubed. Then we fit the derivative as a combination of all candidates, set the small coefficients to zero, and refit, repeating until only a few terms remain. The result is a readable equation, not a black box. For a damped oscillator, SINDy recovers dv over dt equals minus four x minus zero point three v. Noise is the enemy, because differentiation amplifies it, so we smooth first and choose the threshold carefully. A neural ODE, by Chen and colleagues in 2018, takes the opposite approach: it learns the right-hand side with a network, which is flexible but not readable. In this module you will discover the oscillator's law and see when noise breaks it.



The SINDy idea

Write the unknown law as a sparse combination of candidates: dv/dt = Θ(x, v) ξ, where Θ holds library terms as columns and ξ is mostly zero. Sparsity reflects the principle that physical laws are usually simple (parsimony). The method was introduced by Brunton, Proctor and Kutz (2016) and is available in the PySINDy library.

Sequential thresholded least squares

Algorithm (STLSQ): (1) solve least squares for all terms; (2) set coefficients with $|\xi|$ below a threshold λ to zero; (3) refit using only the remaining terms; (4) repeat until nothing changes. In the lab, with noise 0.01 and $\lambda = 0.1$, it finds $-4.00\,x - 0.30\,v$ exactly. With $\lambda = 0.5$ it deletes the damping term ($-3.89\,x$); with $\lambda = 0.01$ junk terms such as $0.02\,x^3$ survive.

TRY Try it

Raise the noise to 0.05. Can you find a threshold that still recovers exactly two terms?

Noise and derivatives

Differentiation turns small noise into large errors, because it divides differences by a small Δt. Defences: smooth the data before differencing (as in the lab), use total-variation regularised derivatives, or use the weak (integral) form of SINDy, which avoids derivatives altogether. Always check a discovered law by simulating it and comparing with held-out data.

Neural ODEs

A neural ODE (Chen et al. 2018) models $dx/dt = f_\theta(x, t)$ with a network and trains it by integrating with an ODE solver and comparing trajectories with data. It handles irregular sampling and complex dynamics, but the learned f is a black box. Universal differential equations (Rackauckas et al. 2020) combine the two: known physics plus a small network for the unknown part, which SINDy can then turn into a formula.

RESEARCH Research angle

Discovering traffic or crowd dynamics from trajectory data with SINDy is an active area: can we find car-following laws directly from vehicle trajectories?

LAB SINDy from scratch

`labs/go2/m24_sindy.py`

Simulate, add noise, smooth, differentiate, and run sequential thresholded least squares.

```
noise 0.001, threshold 0.1: dv/dt = -4.00 x -0.30 v
noise 0.010, threshold 0.1: dv/dt = -4.00 x -0.30 v
noise 0.050, threshold 0.1: dv/dt = -4.02 x -0.29 v
noise 0.050, threshold 0.5: dv/dt = -3.89 x
true law:                       dv/dt = -4.00 x -0.30 v
```

Takeaways

- SINDy finds a law as a sparse combination of candidate terms.
- Sequential thresholded least squares fits, cuts small terms and refits.
- Noise and the threshold decide whether the right terms survive.
- Neural ODEs learn the right-hand side with a network; universal DEs combine both.

Check yourself

1. SINDy assumes that laws are…
   (a) Long and complex · (b) Sparse: made of a few terms · (c) Linear only · (d) Random
2. A threshold that is too high will…
   (a) Keep junk terms · (b) Delete real terms · (c) Have no effect · (d) Add noise
3. Why smooth data before differentiating?
   (a) To speed up code · (b) Differentiation amplifies noise · (c) SINDy requires it by law · (d) To add terms
4. A neural ODE learns…
   (a) A list of words · (b) The right-hand side dx/dt with a network · (c) Only initial conditions · (d) The time step

Answers: 1 (b) Parsimony.; 2 (b) Real small terms vanish.; 3 (b) Noise blows up.; 4 (b) f in dx/dt = f(x).

Words

- **SINDy**: Sparse Identification of Nonlinear Dynamics: discovering equations by sparse regression on a library of terms.
- **Candidate library**: The set of possible terms an equation might contain.
- **Sparse regression**: Regression that drives most coefficients to exactly zero.
- **Neural ODE**: A differential equation whose right-hand side is a neural network.
- **Universal differential equation**: A model combining known physics with a neural network for unknown terms.

Reading

- Brunton, Proctor and Kutz (2016): Discovering governing equations from data: The SINDy paper. https://doi.org/10.1073/pnas.1517384113
- PySINDy documentation: Python package with tutorials. https://pysindy.readthedocs.io/
- Chen et al. (2018): Neural ordinary differential equations: The neural ODE paper. https://arxiv.org/abs/1806.07366

2.5 Operator learning: DeepONet and neural operators

50 minutes · Build

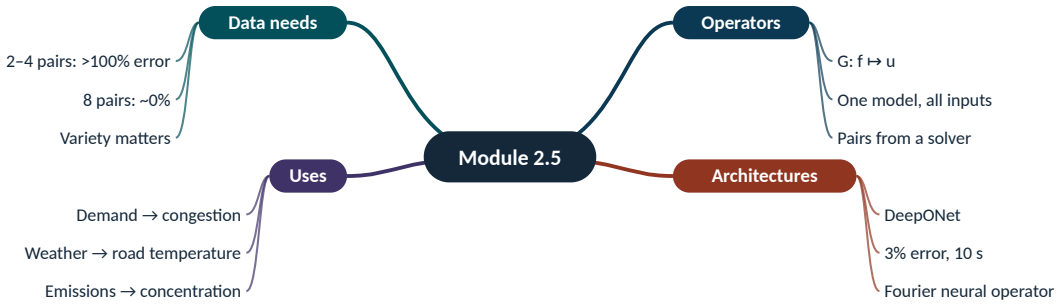
An operator maps a whole input function to a whole output function. Once trained on example pairs, a neural operator solves new cases of a PDE in milliseconds, across many inputs at once.

By the end you can

- explain the difference between learning a function and learning an operator
- describe the branch–trunk structure of DeepONet and the idea of Fourier neural operators
- show why the number and variety of training pairs matter
- train a small DeepONet in PyTorch

Overview

A PINN solves one problem: one set of boundary conditions, one source, one parameter value. Change the input and you must train again. Operator learning removes that limitation. An operator is a map from a whole function to a whole function: from a heat source pattern to the temperature field it creates, or from a traffic demand profile to the resulting congestion pattern. We generate example pairs with a classical solver, then train a neural operator to reproduce the map. Afterwards, new inputs are solved almost instantly. DeepONet, by Lu and colleagues, uses two networks: a branch network reads the input function at sensor points, a trunk network reads the location where we want the answer, and their dot product gives the output. Fourier neural operators, by Li and colleagues, learn in frequency space and can work at different grid resolutions. The catch: the training inputs must cover the kinds of inputs you will meet later. In this module you will learn a solution operator from a few example pairs, and see exactly how many you need.



Functions versus operators

A network usually learns a **function**: numbers in, numbers out. An **operator** takes a function in and gives a function out: G: f ↦ u. For −u″ = f with u(0) = u(1) = 0, G is the solution operator of the Poisson problem. Learning G means one trained model solves the problem for *every* source f, not just one.

How many examples?

In the explorer the inputs are built from 8 sine shapes, so the operator restricted to them has 8 degrees of freedom. With 2 or 4 training pairs the error on new inputs is over 100%; with 8 or more it drops to essentially zero. Real input families are richer, so neural operators often need hundreds to thousands of pairs; the variety of the pairs matters as much as their number.

TRY Try it

Increase training pairs from 1 to 10. At which number does the error collapse? Does adding more grid points change that number?

DeepONet and Fourier neural operators

DeepONet (Lu et al. 2021): $u(x) \approx \Sigma_k\, b_k(f)\, t_k(x)$, branch b, trunk t. The PyTorch lab trains one on 500 pairs in about 10 seconds and reaches about 3% error on new inputs. **Fourier neural operators** (Li et al. 2021) apply learned filters in frequency space; they are resolution-independent and excel at fluid-like problems. Both can be made physics-informed by adding residual terms (PI-DeepONet).

MODEL Lab note

The browser lab learns a linear operator with ridge regression, the simplest operator learner. The PyTorch lab trains a real DeepONet on the same problem.

Transport uses

Operator learning can map demand profiles to congestion patterns along a corridor, weather fields to road-surface temperature, or emissions maps to concentration fields, giving fast surrogates for scenario studies and real-time control. The training data come from validated simulators, so their limits carry over.

LAB Learning a linear solution operator

`labs/go2/m25_operator.py`

A classical solver makes pairs; ridge regression learns the operator matrix.

```
2 training pairs: relative error on new inputs 1.194
 4 training pairs: relative error on new inputs 1.780
 8 training pairs: relative error on new inputs 0.000
12 training pairs: relative error on new inputs 0.000
```

LAB A small DeepONet (PyTorch)

`labs/go2/m25_deeponet_torch.py`

Branch and trunk networks trained on 500 pairs. Requires pip install torch; about 10 seconds.

```
iteration 4000: loss 5.90e-04
relative error on 100 new inputs: 0.027
```

Takeaways

- An operator maps whole functions to whole functions.
- Neural operators are trained on pairs from a classical solver, then solve new inputs instantly.
- DeepONet uses branch and trunk networks; Fourier neural operators learn in frequency space.
- Training inputs must cover the inputs you will meet later.

Check yourself

1. An operator maps…
   (a) A number to a number · (b) A function to a function · (c) A word to a word · (d) An image to a label
2. In DeepONet, the trunk network takes…
   (a) The input function · (b) The query location · (c) The loss · (d) The time step
3. Where do operator-learning training pairs usually come from?
   (a) Random guesses · (b) A classical solver run on many inputs · (c) The test set · (d) A survey
4. A neural operator may fail when…
   (a) Inputs resemble the training inputs · (b) New inputs are unlike any training input · (c) The grid is fine · (d) It is trained longer

Answers: 1 (b) Function in, function out.; 2 (b) Branch: function; trunk: location.; 3 (b) Solver-generated pairs.; 4 (b) Coverage matters.

Words

- **Operator**: A mapping from one function to another, such as from a PDE's source to its solution.
- **DeepONet**: A neural operator with a branch network for the input function and a trunk network for the query location.
- **Fourier neural operator**: A neural operator that applies learned filters in Fourier (frequency) space.
- **Solution operator**: The map from a problem's inputs to its solution.
- **Resolution independence**: The ability to evaluate on grids different from those used in training.

Reading

- Lu et al. (2021): Learning nonlinear operators via DeepONet: The DeepONet paper. https://doi.org/10.1038/s42256-021-00302-5
- Li et al. (2021): Fourier neural operator for parametric PDEs: The FNO paper. https://arxiv.org/abs/2010.08895
- Kovachki et al. (2023): Neural operator: learning maps between function spaces: A comprehensive review. https://www.jmlr.org/papers/v24/21-1524.html

2.6 Gaussian processes and uncertainty

45 minutes · Build

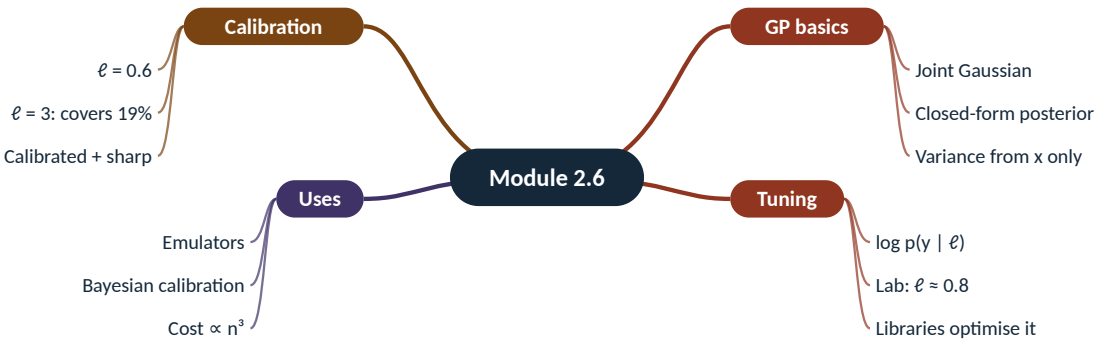
A Gaussian process predicts a value and its uncertainty, widening its error bars far from data. Kernels encode smoothness; marginal likelihood tunes them; calibrated uncertainty is the goal.

By the end you can

- explain a Gaussian process as a distribution over smooth functions
- describe the role of the kernel and its length scale
- check whether uncertainty bands are well calibrated
- tune a length scale by maximum marginal likelihood

Overview

A prediction without an error bar is half an answer. Gaussian processes give both. Think of a Gaussian process as a cloud of possible smooth curves. Before seeing data, the cloud is wide. Each observation pins the curves near that point, and the cloud narrows there. Between observations it stays narrow if they are close and widens if they are far apart. The kernel decides what the curves look like. The squared-exponential kernel says two inputs are similar if they are within about one length scale of each other. A short length scale gives wiggly curves that forget quickly; a long one gives smooth curves that can be overconfident. Instead of guessing, we choose the length scale that makes the data most probable, called maximising the marginal likelihood. The test of good uncertainty is calibration: a 95 percent band should contain the truth about 95 percent of the time. Gaussian processes are the workhorse for surrogates and emulators, Bayesian calibration of models, and deciding where to measure next. In this module you will tune a Gaussian process and check its calibration.



A distribution over functions

A Gaussian process (GP) says any finite set of function values is jointly Gaussian, with covariance given by a kernel. Conditioning on data gives closed-form predictions:

$$\mu(x^*) = k^{*T}\,(K + \sigma_n^2\,I)^{-1}\,y,\quad \sigma^2(x^*) = k(x^*, x^*) - k^{*T}\,(K + \sigma_n^2\,I)^{-1}\,k^*$$

The variance does not depend on y: uncertainty shrinks wherever there are observations, whatever their values.

Length scale and calibration

In the explorer with 8 observations: $\ell = 0.6$ gives an error of about 0.35 with the truth inside the band almost everywhere; $\ell = 3$ is far too smooth (error 1.0, band covers only 19% of the truth: overconfident); $\ell = 0.1$ is wiggly and its band is wide almost everywhere (honest but less useful). Good uncertainty is both **calibrated** (covers as claimed) and **sharp** (as narrow as possible).

TRY Try it

With 6 observations, slide ℓ from 0.1 to 3. Watch the band coverage readout. Where is the band honest and still useful?

Tuning by marginal likelihood

The log marginal likelihood, log p(y | ℓ, σ), balances data fit against complexity automatically. In the lab, scanning ℓ from 0.1 to 3 and picking the maximum gives $\ell \approx 0.8$. Libraries (scikit-learn GaussianProcessRegressor, GPyTorch) optimise it by gradient methods.

Where GPs are used

Emulators of slow simulators (module 3.5, GCAM); **Bayesian calibration** of model parameters with a model-discrepancy term (Kennedy and O'Hagan 2001); **Bayesian optimisation**, choosing the next experiment where improvement is likely; and spatial interpolation of sensor data (kriging). Their cost grows with the cube of the number of points, so sparse approximations are used beyond a few thousand points.

GCAM GCAM link

A GP emulator of a GCAM output gives not only a fast answer but also the uncertainty that tells you where to spend your next GCAM run.

LAB Gaussian process regression from scratch

`labs/go2/m26_gp.py`

Cholesky-based GP prediction, calibration check and length-scale selection by marginal likelihood.

```
length scale 0.1: error 0.699, truth inside 95% band 100%, -log likelihood   2.50
length scale 0.6: error 0.542, truth inside 95% band 99%, -log likelihood  -1.43
length scale 3.0: error 0.799, truth inside 95% band 25%, -log likelihood   5.93
length scale chosen by maximum likelihood: 0.80
```

Takeaways

- A Gaussian process predicts a mean and an uncertainty band.
- The kernel's length scale controls smoothness; bands widen away from data.
- Tune hyperparameters by maximising the marginal likelihood.
- Good uncertainty is calibrated and sharp; GPs power emulators and Bayesian calibration.

Check yourself

1. A Gaussian process's uncertainty is largest…
   (a) At the observations · (b) Far from any observation · (c) At x = 0 · (d) Everywhere equal
2. A very long length scale tends to make a GP…
   (a) Wiggly · (b) Overly smooth and possibly overconfident · (c) Exactly right · (d) Unstable
3. Calibration means a 95% band…
   (a) Is narrow · (b) Contains the truth about 95% of the time · (c) Contains all data · (d) Is 95 units wide
4. GP hyperparameters are commonly tuned by…
   (a) Guessing · (b) Maximising the marginal likelihood · (c) Using the test set · (d) Setting them to 1

Answers: 1 (b) Bands widen away from data.; 2 (b) It assumes far points are similar.; 3 (b) Covers as claimed.; 4 (b) Let the data choose.

Words

- **Gaussian process**: A distribution over functions in which any finite set of values is jointly Gaussian.
- **Kernel**: A function measuring similarity between inputs; it defines a GP's covariance.
- **Length scale**: The distance over which a kernel considers inputs similar.
- **Marginal likelihood**: The probability of the data given the hyperparameters, averaging over functions.
- **Calibration**: Agreement between stated and actual coverage of uncertainty intervals.

Reading

- Rasmussen and Williams (2006): Gaussian Processes for Machine Learning (free): The standard textbook. http://gaussianprocess.org/gpml/
- Distill: A visual exploration of Gaussian processes: Interactive introduction. https://distill.pub/2019/visual-exploration-gaussian-processes/
- Kennedy and O'Hagan (2001): Bayesian calibration of computer models. https://doi.org/10.1111/1467-9868.00294

2.7 Hybrid modelling: physics plus learned corrections

45 minutes · Build

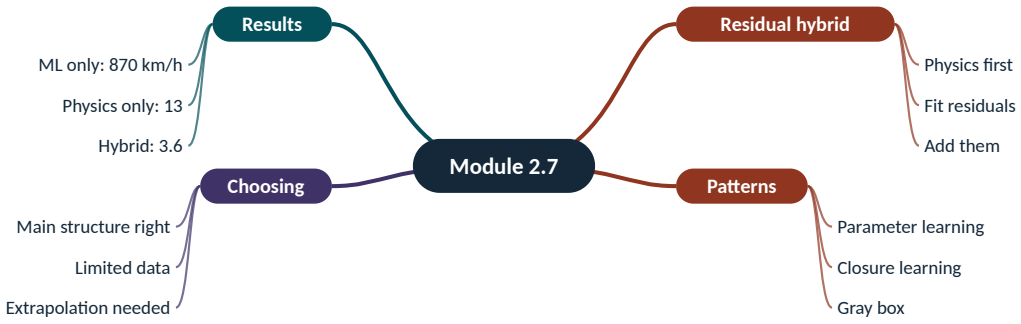
A hybrid model keeps a physics model and learns only what it gets wrong. It needs fewer data than pure machine learning and extrapolates far more safely, as long as the physics part is sound.

By the end you can

- design a residual (additive) hybrid model
- compare physics-only, ML-only and hybrid models beyond the data
- list other hybrid patterns: parameter learning, closures, gray-box models
- decide when a hybrid is the right choice

Overview

Often we have a physics model that is good but not perfect. The textbook Greenshields model says traffic speed falls in a straight line as density rises. Real roads curve away from that line. One option is to throw the physics away and fit a flexible machine learning model to the data. Another is to keep the physics and learn only the correction: the hybrid approach. The learned part now has an easy job, modelling a small, smooth residual, so it needs fewer data and stays well behaved. Crucially, where there are no data, the hybrid falls back on the physics instead of inventing something. In our example, with speed observations only up to sixty vehicles per kilometre, a pure polynomial model is off by hundreds of kilometres per hour at high density, the physics alone is off by about thirteen, and the hybrid by about three and a half. Other hybrid patterns include learning unknown parameters of a physics model, learning closure terms for unresolved processes, and gray-box models. In this module you will compare the three approaches as you change how much data you have.



Residual hybrids, step by step

(1) Fit or choose the physics model, here Greenshields $v = v_f(1 - \rho/\rho_j)$. (2) Compute residuals $r = v_{data} - v_{physics}$. (3) Fit a small, regularised model to the residuals (here a quadratic). (4) Predict $v = v_{physics} + \hat{r}$. Keep the correction small and smooth; a huge learned correction means the physics is wrong, not just imperfect.

Beyond the data

With 20 observations up to 60 veh/km, the errors at higher densities are: ML only (degree-4 polynomial) about 870 km/h, physics only about 13 km/h, hybrid about 3.6 km/h. With data up to 100 veh/km all improve, but the hybrid stays best. The explorer lets you see how quickly the ML-only model goes wrong once it leaves its data.

> **TRY Try it**
>
> Set the highest observed density to 40, then to 120. How do the three errors change? Which model is most stable?

Other hybrid patterns

Parameter learning: a network predicts physical parameters from context (for example, capacity from weather and lighting), then the physics model runs. **Closure learning**: a network represents an unresolved term inside the equation, trained through the solver (universal differential equations). **Gray-box models** mix known and learned blocks in a system diagram. All keep interpretable structure.

When to choose a hybrid

Choose a hybrid when the physics captures the main structure but misses details, data are limited, and extrapolation matters (new conditions, rare events). Avoid it if the physics is badly wrong (the correction will dominate) or if data are abundant and conditions never change (pure ML may be simpler).

> **RESEARCH Research angle**
>
> Hybrid car-following (IDM plus a learned residual) and hybrid traffic-flow models are active topics; see module 3.2.

> **LAB Physics, ML and hybrid beyond the data**
> `labs/go2/m27_hybrid.py`
>
> Fit each model to low-density data and score it at high density.

```
physics only                          error beyond the data:   12.6 km/h
ML only (degree-4 polynomial)         error beyond the data:  868.9 km/h
hybrid: physics + learned correction  error beyond the data:    3.6 km/h
```

Takeaways

- A residual hybrid keeps the physics and learns only its error.
- Hybrids need fewer data and extrapolate far more safely than pure ML.
- Other patterns: parameter learning, closure learning, gray-box models.
- Hybrids suit good-but-imperfect physics, limited data and extrapolation.

Check yourself

1. In a residual hybrid, the machine learning part models…
   (a) The whole output · (b) The difference between data and physics · (c) The noise only · (d) The inputs

2. Beyond the data, a hybrid usually…
   (a) Behaves like the physics model plus a small correction · (b) Goes wild · (c) Returns zero · (d) Copies the last data point
3. Parameter learning means a network…
   (a) Replaces the physics · (b) Predicts the physics model's parameters from context · (c) Learns the loss · (d) Splits data
4. A hybrid is a poor choice when…
   (a) Physics is good but imperfect · (b) The physics is badly wrong · (c) Data are limited · (d) Extrapolation matters

Answers: 1 (b) Learn the gap.; 2 (a) Falls back on physics.; 3 (b) Context → parameters.; 4 (b) The correction would dominate.

Words

- **Hybrid model**: A model combining a physics-based component with a data-driven component.
- **Residual learning**: Learning the error of a physics model and adding it as a correction.
- **Closure term**: A term representing unresolved processes in a coarse model.
- **Gray-box model**: A model mixing known physical structure with learned components.
- **Greenshields model**: A linear speed–density relation for traffic, from 1935.

Reading

- Willard et al. (2022): Integrating scientific knowledge with machine learning: A survey of hybrid patterns. https://doi.org/10.1145/3514228
- Rackauckas et al. (2020): Universal differential equations: Closure learning inside equations. https://arxiv.org/abs/2001.04385
- Greenshields (1935): A study of traffic capacity: The original speed–density study. https://onlinepubs.trb.org/Onlinepubs/hrbproceedings/14/14P1-024.pdf

2.8 Validation, uncertainty and deployment

45 minutes · Deploy

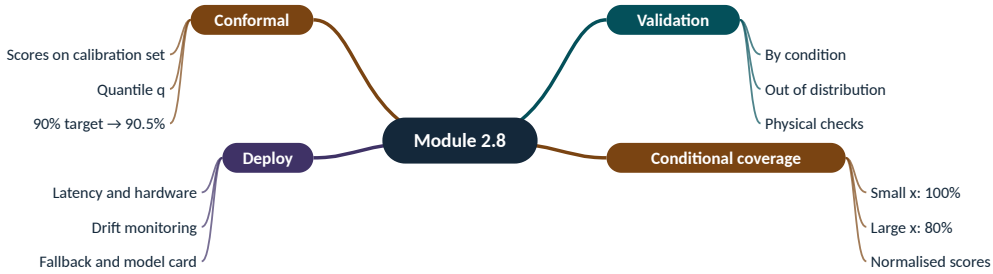
Before a SciML model is used, test it on unseen conditions, check physical consistency, attach calibrated intervals such as conformal predictions, and plan for monitoring, drift and fallbacks in deployment.

By the end you can

- design validation tests beyond a single error number
- build split conformal prediction intervals and check their coverage
- explain conditional versus average coverage
- list deployment concerns: latency, drift, monitoring and fallbacks

Overview

A model is not finished when its training loss is low. Before anyone relies on it, we need four kinds of evidence. First, accuracy on truly unseen conditions: a different week, another city, a holiday. Second, physical consistency: does it conserve vehicles, keep speeds positive, respect capacity? Third, honest uncertainty: every prediction should come with an interval that contains the truth as often as it claims. Split conformal prediction is a simple, powerful way to get such intervals. Take a calibration set the model has not seen, compute its errors, and use the error that is beaten ninety percent of the time as a plus-or-minus band. The guarantee holds on average for any model, but not necessarily for every kind of input: when the noise grows with the input, the band is too wide for small inputs and too narrow for large ones. Fourth, a deployment plan: speed, monitoring for data drift, retraining, and a safe fallback when the model is unsure or sensors fail. In this module you will build conformal intervals and see where they fall short.



Validation beyond one number

Report errors separately for normal days, peaks, incidents and holidays; for each site; and for future periods. Test out of distribution on purpose. Add **physical checks**: count how often predictions break conservation or bounds. A model with slightly higher average error but no physical violations is often the safer choice.

Split conformal prediction

(1) Train the model. (2) On a separate calibration set of n points, compute scores $s_i = |y_i - \hat{y}_i|$. (3) Take q = the $\lceil (n+1)(1-\alpha) \rceil$-th smallest score. (4) Predict $\hat{y} \pm q$. For a new exchangeable point, $P(y \text{ inside}) \geq 1 - \alpha$ on average (Vovk et al. 2005; Angelopoulos and Bates 2023). In the lab with 1,000 calibration points, a 90% target gives 90.5% coverage.

TRY Try it

Use only 10 calibration examples, then 500. How much does the achieved coverage wobble around the target? Look at large inputs.

Average is not conditional

The guarantee is marginal: averaged over all inputs. When noise grows with x, the same ±q band over-covers small x (about 100%) and under-covers large x (about 80%). Remedies: conformalised quantile regression, normalised scores (divide by a predicted spread), or calibrating separately by group. Always check coverage in the subgroups that matter, such as peak hours.

Deployment

Decide latency (an automated vehicle needs milliseconds; a planning study can wait), hardware (edge device or cloud), and monitoring: track input distributions and live errors for **drift**, retrain on a schedule, and define a **fallback** (pure physics model, last good estimate, or a human) when uncertainty is high or sensors fail. Publish a **model card** stating scope, data and limits (Mitchell et al. 2019).

TRAP Common trap

Calibrating on the training set gives intervals that are far too narrow, because the model has already fitted those points.

LAB Split conformal prediction

`labs/go2/m28_conformal.py`

Calibrate on 1,000 points, test on 20,000, and check coverage overall and by input size.

```
target 80%: interval ±1.00, coverage 81.9% (x<1: 97%, x>2: 67%)
target 90%: interval ±1.32, coverage 90.5% (x<1: 100%, x>2: 80%)
target 95%: interval ±1.66, coverage 95.2% (x<1: 100%, x>2: 89%)
```

Takeaways

- Validate on unseen conditions and check physical consistency, not just one error number.
- Split conformal prediction gives distribution-free intervals with average coverage guarantees.
- Average coverage can hide under-coverage in important subgroups.
- Deploy with monitoring for drift, retraining, fallbacks and a model card.

Check yourself

1. Split conformal intervals use errors from…
   (a) The training set · (b) A separate calibration set · (c) The test set · (d) Random numbers

2. The conformal guarantee holds…
   (a) For every input · (b) On average over inputs · (c) Only for linear models · (d) Never
3. When noise grows with x, a constant ±q band…
   (a) Covers all x equally · (b) Under-covers large x and over-covers small x · (c) Is always too wide · (d) Is undefined
4. A fallback in deployment means…
   (a) Deleting the model · (b) A safe alternative when the model is unsure or sensors fail · (c) Retraining every second · (d) Lower accuracy on purpose

Answers: 1 (b) Unseen calibration data.; 2 (b) Marginal coverage.; 3 (b) Conditional coverage fails.; 4 (b) Safe mode.

Words

- **Conformal prediction**: A method that turns any model's errors on calibration data into prediction intervals with coverage guarantees.
- **Marginal coverage**: Coverage averaged over all inputs.
- **Conditional coverage**: Coverage for a specific input or subgroup.
- **Data drift**: A change in the distribution of inputs or outcomes after deployment.
- **Model card**: A short document stating a model's intended use, data, performance and limits.

Reading

- Angelopoulos and Bates (2023): Conformal prediction: a gentle introduction: The clearest tutorial, with code. https://doi.org/10.1561/2200000101
- Mitchell et al. (2019): Model cards for model reporting: How to document a model. https://doi.org/10.1145/3287560.3287596
- NIST AI Risk Management Framework: Guidance on trustworthy AI deployment. https://www.nist.gov/itl/ai-risk-management-framework

Questions for spaced practice

From the question bank: answer from memory, then check. Revisit after 1, 3, 7 and 14 days.

2.1 The SciML workflow and toolchain

1. **Recall:** List the seven steps of the SciML workflow.
   → Frame the question, gather data and physics, split honestly, build a baseline, add learning and physics, validate with uncertainty, document and share.
2. **Explain:** Why does a random split over-estimate performance for time series?
   → Neighbouring hours are very similar, so the model effectively sees the answer for test hours through their neighbours in training.
3. **Apply:** You normalise data using the mean of the whole data set before splitting. Problem?
   → Yes: the test set's statistics leak into training. Fit normalisation on training data only.
4. **Recall:** Name two SciML libraries.
   → DeepXDE, NVIDIA PhysicsNeMo, NeuroMANCER, PySINDy.
5. **Explain:** Why fix random seeds?
   → So training runs can be repeated exactly and differences can be attributed to changes you made.
6. **Apply:** Propose a baseline for next-day hourly flow.
   → The same hour on the same weekday last week (seasonal persistence), or a physics/capacity model alone.

2.2 Build a physics-informed neural network, step by step

1. **Recall:** List the six steps to build a PINN.
   → Network, collocation points, residual by autodiff, composite loss, training with Adam, and (optionally) a trainable parameter for the inverse problem.
2. **Explain:** How do you compute dŷ/dt in PyTorch?
   → Set requires_grad on t, compute y = net(t), then torch.autograd.grad(y, t, ones_like(y), create_graph=True).
3. **Apply:** Data loss is stuck high, physics loss tiny. Diagnose.
   → The physics weight is too large or the network too small; lower λ or enlarge the network.
4. **Explain:** How does a PINN estimate k?
   → k is a trainable parameter; minimising data + residual loss finds the k under which the law and data agree.
5. **Explain:** Why scale inputs and outputs to about ±1?
   → Tanh networks train best when inputs are order one; unscaled times or values make gradients badly balanced.
6. **Apply:** When should you not use a PINN?
   → For a well-posed forward problem with known parameters where a classical solver is faster and more accurate.

2.3 Solving PDEs: classical solvers and PINNs

1. **Recall:** Write the explicit finite-difference update for the heat equation.
   → $u_i \leftarrow u_i + r\,(u_{i+1} - 2u_i + u_{i-1})$, with $r = \alpha\Delta t/\Delta x^2$.
2. **Explain:** What happens if $r > 0.5$?
   → High-frequency errors are amplified every step and the solution blows up.
3. **Calculate:** $\Delta x = 0.025$, $\alpha = 1$. Largest stable explicit Δt?
   → $\Delta t \le 0.5 \times 0.025^2 = 0.0003125$.
4. **Explain:** Why use finite volumes for traffic?
   → They conserve vehicles exactly and handle shock waves without spurious oscillations.
5. **Recall:** What terms make up a PINN loss for the heat equation?
   → PDE residual at interior points, boundary-condition terms and an initial-condition term.

6. **Apply:** When would you choose a PINN over finite differences for a PDE?
   → When fusing sparse data, inferring unknown parameters, or handling complex geometries without meshing.

2.4 Discovering equations from data: SINDy and neural ODEs

1. **Recall:** What does SINDy stand for?
   → Sparse Identification of Nonlinear Dynamics.
2. **Explain:** Describe the STLSQ algorithm.
   → Least squares on all library terms, zero those below a threshold, refit the rest, repeat until stable.
3. **Apply:** SINDy returns $dv/dt = -4.0x - 0.3v + 0.02x^3 + 0.01$. What do you change?
   → Raise the threshold (and/or smooth the data) to remove small spurious terms, then check by simulation.
4. **Explain:** Why is noise a particular problem for equation discovery?
   → Derivatives are estimated from differences divided by small time steps, which magnifies noise.
5. **Explain:** When might you prefer a neural ODE to SINDy?
   → When the law is too complex for a small library or interpretability is less important than accuracy.
6. **Apply:** Suggest candidate terms for discovering a car-following law.
   → Gap, relative speed, own speed, their products and squares, and terms like 1/gap.

2.5 Operator learning: DeepONet and neural operators

1. **Recall:** What is the difference between a function and an operator?
   → A function maps numbers to numbers; an operator maps a whole function to another function.
2. **Explain:** Describe the DeepONet architecture.
   → A branch net encodes the input function sampled at sensors; a trunk net encodes the query location; their dot product gives the output.
3. **Explain:** Why does operator learning pay off compared with PINNs for many scenarios?
   → A PINN is retrained for each input; a trained operator solves new inputs instantly.
4. **Apply:** Your FNO works on training demand patterns but fails on a new event day. Why?
   → The event-day input is outside the training distribution; add similar examples or use physics-informed training.
5. **Recall:** Name two neural operator architectures.
   → DeepONet and the Fourier neural operator.
6. **Apply:** Suggest a transport operator-learning task.
   → Demand profile → congestion pattern on a corridor; emissions map → pollutant concentration.

2.6 Gaussian processes and uncertainty

1. **Recall:** What two things does a GP prediction provide?
   → A mean (best estimate) and a variance (uncertainty) at every input.
2. **Explain:** What does the kernel length scale control?
   → How far apart two inputs can be and still be considered similar, and hence how smooth the functions are.
3. **Explain:** Why does GP variance not depend on observed y values?
   → With fixed hyperparameters, the posterior variance depends only on where observations are, not on their values.
4. **Apply:** Your 95% bands cover only 60% of held-out points. Diagnose.
   → Overconfident: the length scale may be too long or the noise too small; re-tune by marginal likelihood or rethink the kernel.
5. **Recall:** Name two uses of GPs in SciML.
   → Emulators of slow simulators, Bayesian calibration, Bayesian optimisation, kriging of sensor data.
6. **Explain:** Why do GPs become slow for large data sets?
   → Exact inference needs a matrix factorisation costing order n^3 operations.

2.7 Hybrid modelling: physics plus learned corrections

1. **Recall:** Write the form of a residual hybrid model.
   → $y = \text{physics}(x) + \text{ML}(x)$, where ML is trained on data − physics.
2. **Explain:** Why does a hybrid extrapolate better than pure ML?
   → Outside the data the small correction stays modest, so the prediction follows the physics, which is valid there.
3. **Apply:** Your learned correction is as large as the physics prediction. What does this mean?
   → The physics model is badly wrong or mis-specified; revisit it before trusting the hybrid.
4. **Recall:** Name three hybrid patterns.
   → Residual learning, parameter learning, closure learning (and gray-box models).
5. **Apply:** Design a parameter-learning hybrid for road capacity.
   → A network predicts capacity and free-flow speed from weather, lighting and time; a traffic-flow model uses them.
6. **Explain:** When might pure ML be fine instead?
   → When data are abundant, cover all conditions of interest, and extrapolation is not needed.

2.8 Validation, uncertainty and deployment

1. **Recall:** List four kinds of evidence needed before deploying a SciML model.
   → Accuracy on unseen conditions, physical consistency, calibrated uncertainty, and a deployment plan with monitoring and fallbacks.
2. **Explain:** Describe split conformal prediction.
   → Compute absolute errors on a calibration set, take the $\lceil (n+1)(1-\alpha) \rceil$-th smallest as q, and predict $\hat{y} \pm q$.
3. **Explain:** Why can conformal intervals under-cover some inputs?
   → The guarantee is averaged over inputs; with input-dependent noise, a constant band is too narrow where noise is large.
4. **Apply:** Your traffic model's intervals cover 90% overall but 70% at peak hours. Fix?
   → Use normalised scores or conformalised quantile regression, or calibrate peak hours separately.
5. **Recall:** What is data drift?
   → A change in input or outcome distributions after deployment, which degrades model performance.
6. **Explain:** Why publish a model card?
   → So users know the model's intended use, training data, performance and where it must not be used.